

Introduction to Advanced Data Types in Python

Python provides several advanced (built-in) data types to store collections of data. These types are more powerful and flexible than basic types.

The main advanced data types are : **List**, **Tuple**, **Dictionary**, **Set**.

1. **LIST** A list is an ordered, mutable (changeable) collection of items.
Items can be of different data types.

Syntax :

```
my_list = [item1, item2, item3, ...]
```

Example :

```
nums = [10, 20, 30, 'hello', 4.5]
print(nums[0])    # 10
print(nums[3])    # hello
```

Operations / Methods :

- **Append()** : add item at end
- **Insert(i, x)** : insert x at index i
- **Remove(x)** : remove first occ. of x
- **Pop(i)** : remove item at index i
- **Sort()** : sort the list
- **Len()** : number of items

Index :	0	1	2	3	4
	10	20	30	'hello'	4.5
Negative Index :	-5	-4	-3	-2	-1

2. **TUPLE** A tuple is an ordered, immutable (unchangeable) collection of items.
Once created, its elements cannot be changed.

Syntax :

```
my_tuple = (item1, item2, item3, ...)
```

Example :

```
days = ('Mon', 'Tue', 'Wed', 'Thu')
print(days[1])    # Tue
print(len(days))  # 4
```

Operations :

- **Count(x)** : count occurrences of x
 - **Index(x)** : first index of x
 - **Len()** : number of items
- (Note: No append, insert, remove etc.)

3. **DICTIONARY** A dictionary is an unordered collection of key-value pairs.
Each key must be unique and immutable.

Syntax :

```
my_dict = {key1: value1, key2: value2, ...}
```

Example :

```
student = {'name': 'Amit', 'age': 20, 'grade': 'A'}
print(student['name']) # Amit
print(student['age'])  # 20
```

Operations / Methods :

- **Keys()** : return all keys
- **Values()** : return all values
- **Items()** : return key-value pairs
- **Get(k)** : return value for key k
- **Update()** : update dictionary

Key Value

'name'	'age'	'grade'
'Amit'	20	'A'

Note :

Accessing a non-existing key using [] gives **KeyError**.
Use **get()** method to avoid error.

4. **SET** A set is an unordered collection of unique elements.
It does not allow duplicate values.

Syntax :

```
my_set = {item1, item2, item3, ...}
```

Example :

```
s = {1, 2, 3, 4, 4, 3}
# duplicates removed
print(s)    # {1, 2, 3, 4}
```

Operations / Methods :

- **Add(x)** : add element x
- **Remove(x)** : remove element x
- **Discard(x)** : remove x if present
- **Union(s2)** : elements in both sets
- **Intersection(s2)** : common elements

Set Operations

Examples :

```
A = {1, 2, 3}
B = {3, 4, 5}
A ∪ B = {1, 2, 3, 4, 5}
A ∩ B = {3}
```

Summary :

1. List → ordered, mutable, allows duplicate items.
2. Tuple → ordered, immutable, allows duplicate items.
3. Dictionary → unordered key-value pairs, keys are unique.
4. Set → unordered, unique elements only.

Why use these?

- To store and manage collections of data efficiently.
- To perform operations like search, insert, delete, update and mathematical set operations.

Lists - creating, accessing, modifying, slicing and iterating operations on List.

A list is an ordered, mutable (changeable) collection of items. It can store elements of different data types.

Syntax : `my_list = [item1, item2, item3, ...]`

Example : `marks = [85, 90, 78, 92, 88]` # list of student marks

1. Creating a List

- Lists are created using square brackets [].
- Elements are separated by commas.

Examples :

```
a = [] # empty list
b = [1, 2, 3, 4] # list of numbers
c = ['a', 'b', 'c'] # list of strings
d = [10, 'hello', 3.14, True] # mixed list
```

2. Accessing Elements in a List

- Elements are accessed using index.
- Indexing starts from 0.

Example :

`marks = [85, 90, 78, 92, 88]`

Index :	0	1	2	3	4
	85	90	78	92	88

- `marks[0]` → 85 (first element)
- `marks[2]` → 78 (third element)
- `marks[-1]` → 88 (last element)
- `marks[-2]` → 92 (second last element)

3. Modifying (Changing) Elements in a List

- Since lists are mutable, we can change, add or remove elements.

Example : `marks = [85, 90, 78, 92, 88]`

```
marks[1] = 95 # change element at index 1
marks[3] = 100 # change element at index 3
marks.append(93) # add element at end
marks.insert(2, 80) # insert 80 at index 2
marks.remove(78) # remove value 78
del marks[0] # delete element at index 0
```

Result :

```
marks = [85, 95, 78, 92, 88]
marks = [85, 95, 78, 100, 88]
marks = [85, 95, 78, 100, 88, 93]
marks = [85, 95, 80, 78, 100, 88, 93]
marks = [85, 95, 80, 100, 88, 93]
marks = [95, 80, 100, 88, 93]
```

4. Slicing a List

- Slicing returns a part of the list.
- Syntax : `list[start : end : step]`

Example : `nums = [10, 20, 30, 40, 50, 60]`

Slicing	Result
<code>nums[1:4]</code>	<code>[20, 30, 40]</code>
<code>nums[:4]</code>	<code>[10, 20, 30, 40]</code>
<code>nums[2:]</code>	<code>[30, 40, 50, 60]</code>
<code>nums[:2]</code>	<code>[10, 20]</code> # step = 1
<code>nums[::-1]</code>	<code>[60, 50, 40, 30, 20, 10]</code> # reverse

Note :

- start is inclusive
- end is exclusive
- default start = 0
- default end = last index + 1
- default step = 1

5. Iterating (Traversing) a List

- We can iterate through list using loop.

Example 1 : Using for loop

```
fruits = ['apple', 'banana', 'mango']
for fruit in fruits:
    print(fruit)
```

Output :
apple
banana
mango

Example 2 : Using while loop

```
i = 0
while i < len(fruits):
    print(fruits[i])
    i += 1
```

Output :
apple
banana
mango

★ Lists are powerful because they are ordered, mutable and can store multiple types of data. They are widely used in real world programming.



Lists - Creating

A list in Python is an ordered, mutable (changeable) collection of items.

Lists are created using square brackets `[]` and elements are separated by commas.

1. What is a List ?

- It is an ordered sequence of elements.
- It can store elements of different data types.
- It is mutable (we can change, add or remove elements).
- Duplicates are allowed.

2. Creating a List

In Python, we can create a list in several ways.

① Method 1 : Creating an empty list

```
my_list = [] # empty list
```

Example :

```
my_list = []
```

Output :

```
[]
```

② Method 2 : Creating a list with elements

```
my_list = [item1, item2, item3, ...] # elements  
                                     separated  
                                     by commas
```

Example :

```
numbers = [10, 20, 30, 40, 50]
```

Output :

```
[10, 20, 30, 40, 50]
```

③ Method 3 : Creating a list with different data types

```
my_list = [10, 'Hello', 3.14, True] # int, string,  
                                     float, boolean
```

Example :

```
mixed = [1, 'Python', 3.5, False]
```

Output :

```
[1, 'Python', 3.5, False]
```

④ Method 4 : Creating a list using the list() constructor

```
my_list = list(iterable)  
# iterable can be string, range, tuple etc.
```

Examples :

```
a = list('ABC') # from string  
b = list(1, 2, 3) # from tuple  
c = list(range(1, 5)) # from range
```

Output :

```
a = ['A', 'B', 'C']  
b = [1, 2, 3]  
c = [1, 2, 3, 4]
```

⑤ Method 5 : Using list comprehension

```
squares = [x*x for x in range(1, 6)] # create list of  
                                     squares
```

Example :

```
squares = [x*x for x in range(1, 6)]
```

Output :

```
[1, 4, 9, 16, 25]
```

Note :

- Lists are mutable, so we can modify them after creation.
- Elements in a list are indexed. Index starts from 0.
- A list can store duplicate values.

Example of duplicate values :

```
dup_list = [1, 2, 2, 3, 4, 4, 5]
```

Output : [1, 2, 2, 3, 4, 4, 5]

★ Summary :

- Use `[]` to create a list.
- Elements are separated by commas.
- Lists can be empty or contain elements.
- We can create lists in multiple ways.



Lists - Accessing

Elements of a list are accessed using index. Indexing starts from 0.

Syntax :

`list_name [index]`

Where,

list_name → name of the list

index → position of the element (starts from 0)

Example :

`marks = [85, 90, 78, 92, 88]`

1. Accessing elements using Positive Index

Index	0	1	2	3	4
Element	85	90	78	92	88

Accessing :

- `marks[0]` → 85 # first element
- `marks[2]` → 78 # third element
- `marks[4]` → 88 # last element

2. Accessing elements using Negative Index

Index	-5	-4	-3	-2	-1
Element	85	90	78	92	88

Accessing :

- `marks[-1]` → 88 # last element
- `marks[-2]` → 92 # second last element
- `marks[-5]` → 85 # first element

3. Accessing elements from different data types in a list

Example :

`data = ['Python', 3.14, True, 25]`

Index	0	1	2	3
Element	'Python'	3.14	True	25

Accessing :

- `data[0]` → 'Python'
- `data[1]` → 3.14
- `data[2]` → True
- `data[3]` → 25

Note :

If we try to access an index which does not exist, Python will show `IndexError`.

Example : `marks[5]` → `IndexError`

4. Accessing using a variable

Example :

```
i = 2 # index stored in a variable
marks = [85, 90, 78, 92, 88]
print(marks[i]) # Output: 78
```

Try it Yourself

`lst = [10, 20, 30, 40, 50]`

Find → `lst[0]`, `lst[2]`, `lst[4]`

`lst[-1]`, `lst[-3]`

Summary :

- We access list elements using index.
- Indexing starts from 0.
- Negative index is used to access elements from the end.
- If index is out of range, `IndexError` occurs.

Remember :

Valid index range for a list of length n is : $-(n)$ to $(n-1)$

Lists - Modifying

Lists are mutable, which means we can change, add or remove elements after the list is created.

Syntax : `list_name.method()` or `list_name[index] = new_value`

Example List : `marks = [85, 90, 78, 92, 88]`

(Original list) Index : 0 1 2 3 4

1. Changing (Updating) an element

We can change the value of an existing element using index.

Example :

```
marks[2] = 80      # change 78 to 80
print(marks)      # [85, 90, 80, 92, 88]
```

2. Adding an element

We can add elements to the list.

- `append()` - adds element at the end.
- `insert(index, value)` - adds element at specific position.
- `extend(iterable)` - adds multiple elements (from another list/iterable) at the end.

Example :

```
marks.append(95)      # [85, 90, 78, 92, 88, 95]
marks.insert(2, 76)      # [85, 90, 76, 78, 92, 88]
marks.extend([70, 65])      # [85, 90, 78, 92, 88, 70, 65]
```

3. Removing elements

We can remove elements from the list.

- `remove(value)` - removes first occurrence of value.
- `pop(index)` - removes and returns element at index (default is last element).
- `del list[index]` - deletes element at index.
- `clear()` - removes all elements from the list.

Example :

```
marks.remove(92)      # [85, 90, 78, 88]
x = marks.pop()      # x = 88, list → [85, 90, 78]
marks.pop(1)      # removes 90, list → [85, 78]
del marks[0]      # deletes 85, list → [78]
marks.clear()      # [] (empty list)
```

4. Inserting (Adding at specific position)

We can insert an element at a particular index.

Example :

```
marks = [85, 90, 78, 92, 88]
marks.insert(2, 76)      # insert 76 at index 2
# Result → [85, 90, 76, 78, 92, 88]
```

5. Changing multiple elements (using slice assignment)

We can replace a range (slice) of elements with new values.

Example :

```
marks = [85, 90, 78, 92, 88]
marks[1:3] = [95, 100]      # replace index 1 to 2
# Result → [85, 95, 100, 92, 88]
```

6. Sorting the list

We can sort the list (ascending by default).

Example :

```
marks = [85, 90, 78, 92, 88]
marks.sort()      # [78, 85, 88, 90, 92]
marks.sort(reverse=True)      # [92, 90, 88, 85, 78]
```

7. Reversing the list

We can reverse the order of elements.

Example :

```
marks = [85, 90, 78, 92, 88]
marks.reverse()      # [88, 92, 78, 90, 85]
```

Summary :

- We can modify lists after creation.
- We can add, remove, change or reorder elements.
- List methods make modification easy and flexible.

★ Common methods used for modifying lists

<code>append()</code>	<code>insert()</code>	<code>extend()</code>	<code>remove()</code>
<code>pop()</code>	<code>del</code>	<code>clear()</code>	<code>sort()</code> <code>reverse()</code>

Lists - Slicing

Slicing means taking a part (sub-list) of a list.

We use the syntax: `list_name[start : end : step]`

Syntax :

`list_name[start : end : step]`

Where,

- start → starting index (inclusive)
- end → ending index (exclusive)
- step → jump/step value (default is 1)

Example List : `nums = [10, 20, 30, 40, 50, 60]`

Index :	0	1	2	3	4	5
Index	0	1	2	3	4	5
Value	10	20	30	40	50	60

Note :

- start is inclusive (included)
- end is exclusive (not included)
- default start = 0
- default end = last index + 1
- default step = 1

1. Basic Slicing (step = 1)

Expression	Result	Explanation
<code>nums[0:3]</code>	<code>[10, 20, 30]</code>	from index 0 to 2
<code>nums[1:4]</code>	<code>[20, 30, 40]</code>	from index 1 to 3
<code>nums[:4]</code>	<code>[10, 20, 30, 40]</code>	from beginning to index 3
<code>nums[2:]</code>	<code>[30, 40, 50, 60]</code>	from index 2 to end
<code>nums[:]</code>	<code>[10, 20, 30, 40, 50, 60]</code>	whole list (shallow copy)

2. Slicing with Step

Expression	Result	Explanation
<code>nums[::2]</code>	<code>[10, 30, 50]</code>	every 2nd element from start
<code>nums[1::2]</code>	<code>[20, 40, 60]</code>	every 2nd element from index 1
<code>nums[::3]</code>	<code>[10, 40]</code>	every 3rd element from start
<code>nums[::-1]</code>	<code>[60, 50, 40, 30, 20, 10]</code>	reverse the list

3. Negative Indexing in Slicing

Expression	Result	Explanation
<code>nums[-3:]</code>	<code>[40, 50, 60]</code>	last 3 elements
<code>nums[:-2]</code>	<code>[10, 20, 30, 40]</code>	all except last 2
<code>nums[-4:-1]</code>	<code>[30, 40, 50]</code>	from -4 to -2

Key Points

- Slicing does not change the original list.
- It returns a new sub-list.
- Very useful for working with parts of a list.

Try It Yourself

Let: `data = [5, 15, 25, 35, 45, 55, 65]`

- `data[2:5]` → _____
- `data[:4]` → _____
- `data[::2]` → _____
- `data[1::3]` → _____
- `data[::-1]` → _____
- `data[-3:]` → _____

Summary

- Use slicing to extract a sub-list.
- Format: `list[start : end : step]`
- Defaults: `start=0, end=last+1, step=1`

Lists - Iterating Operations

Iterating means accessing each element of a list one by one.

Python provides different ways to iterate (traverse) through a list.

1. Iterating using for loop

Example :

```
fruits = ['apple', 'banana', 'mango', 'orange']
# Iterating through list elements using for loop
for fruit in fruits :
    print(fruit)
```

Output :

```
apple
banana
mango
orange
```

Explanation :

The loop variable 'fruit' takes each value from the list one by one.

2. Iterating using range() and index

Example :

```
nums = [10, 20, 30, 40, 50]
# Accessing elements using index
for i in range(len(nums)) :
    print(i, nums[i])
```

Output :

```
0 10
1 20
2 30
3 40
4 50
```

Explanation :

Here, we are using index to access each element of the list.

3. Iterating using enumerate()

Example :

```
cities = ['Delhi', 'Mumbai', 'Kolkata', 'Chennai']
# Using enumerate() to get index and value together
for index, city in enumerate(cities) :
    print(index, city)
```

Output :

```
0 Delhi
1 Mumbai
2 Kolkata
3 Chennai
```

Explanation :

enumerate() returns both index and value in each iteration.

4. Iterating using while loop

Example : numbers = [5, 15, 25, 35, 45]

```
i = 0 # initialize index
while i < len(numbers) : # loop until last index
    print(numbers[i])
    i = i + 1 # update index
```

Output :

```
5
15
25
35
45
```

Explanation :

We use while loop with index to traverse the list.

Other useful iterative operations on lists

- Iterating in reverse order

```
for item in reversed(fruits) :
    print(item)
```

Output : orange, mango, banana, apple

- Iterating over a copy of list

```
for item in fruits.copy() :
    print(item)
```

Output : apple, banana, mango, orange

- Iterating with condition (filtering)

```
for num in numbers :
    if num > 20 :
        print(num)
```

Output : 25, 35, 45

Note :

Iteration helps us process each element of the list efficiently. It is widely used in real-world programs.

